

TREES DATA STRUCTURES

Fundamentals, Balanced Trees & Advanced Structures

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-I



Course Outline

- 1 Tree Structures Fundamentals
- 2 Balanced Trees
- 3 Advanced Tree Structures



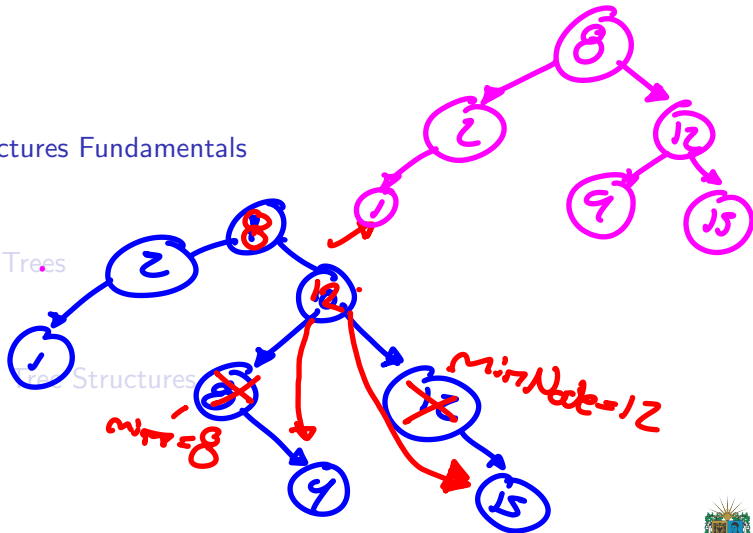
Outline

0-7

1 Tree Structures Fundamentals

2 Balanced Trees

3 Advanced Tree Structures



Tree Terminology

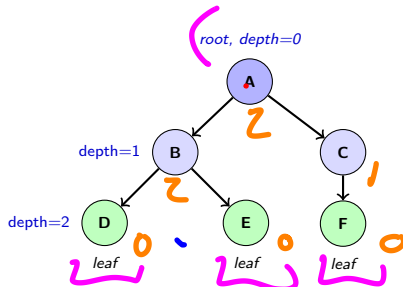
Definition

A **tree** is a hierarchical, **acyclic** structure: ~~nodes~~ connected by directed edges where every non-root node has exactly **one** parent.

↓ arrow

Key terms

- **Root** — top node, no parent
- **Leaf** — node with no children
- **Depth** — distance from root
- **Height** — max depth in (sub)tree
- **Degree** — number of children



Tree vs. List

A list is a **degenerate** tree where every node has degree ≤ 1 .

Height = 2

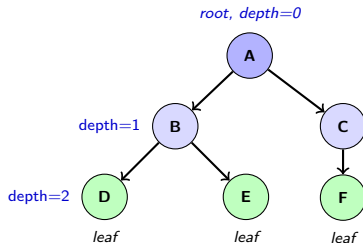
Tree Terminology

Definition

A **tree** is a hierarchical, **acyclic** structure: nodes connected by directed edges where every non-root node has exactly **one parent**.

Key terms

- **Root** — top node, no parent
- **Leaf** — node with no children
- **Depth** — distance from root
- **Height** — max depth in (sub)tree
- **Degree** — number of children



Tree vs. List

A list is a **degenerate** tree where every node has degree ≤ 1 .

Binary Tree: C Structure

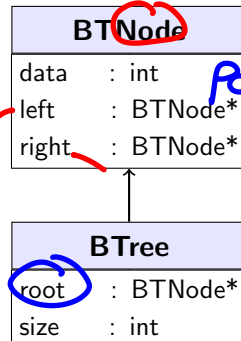
Definition

A **binary tree** has each node with at most **two children**: left and right.

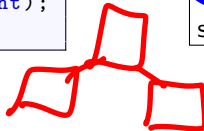
```
typedef struct BTreeNode {
    int data;
    struct BTreeNode *left;
    struct BTreeNode *right;
} BTreeNode;

typedef struct {
    BTreeNode *root;
    int size;
} BTree;

BTreeNode *bt_insert(BTree*, int);
BTreeNode *bt_search(BTree*, int);
void bt_free (BTree*);
```



Pointers



Binary Search Tree (BST) I

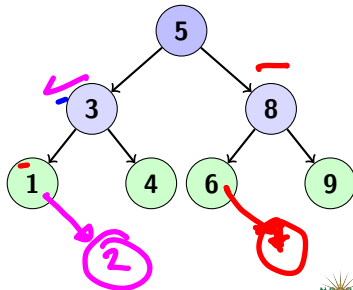
BST property

For every node n : all keys in left subtree $< n <$ all keys in right subtree.

```

BTNode *bt_insert(BTNode *n, int v){
    if (!n) {
        BTNode *p = malloc(sizeof(*p));
        p->data = v;
        p->left = p->right = NULL;
        return p;
    }
    if (v < n->data)
        n->left = bt_insert(n->left, v);
    else if (v > n->data)
        n->right = bt_insert(n->right, v);
    return n;
}

```

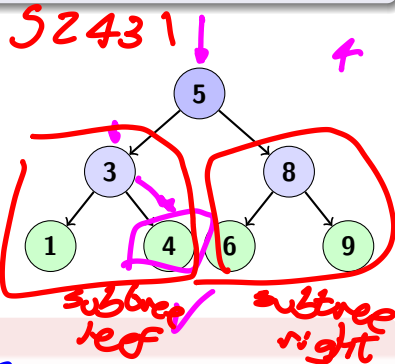


Binary Search Tree (BST) II

BST property

For every node n : all keys in **left** subtree $< n <$ all keys in **right** subtree.

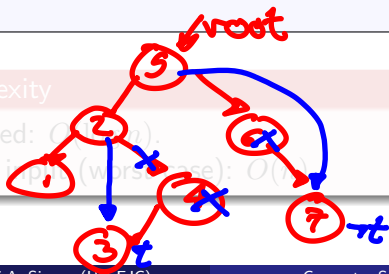
```
BTNode *bt_search(BTNode *n, int v)
{
    if (!n || n->data == v)
        return n;
    if (v < n->data)
        return bt_search(n->left, v);
    return bt_search(n->right, v);
}
```



Complexity

Balanced: $O(\log n)$

Sorted input (worst case): $O(n)$

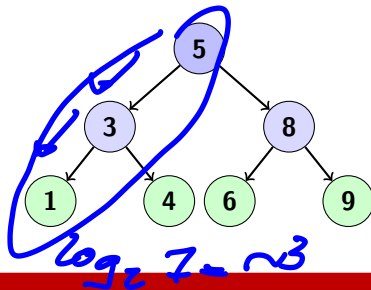


Binary Search Tree (BST) II

BST property

For every node n : all keys in **left** subtree $< n <$ all keys in **right** subtree.

```
BTNode *bt_search(BTNode *n, int v)
{
    if (!n || n->data == v)
        return n;
    if (v < n->data)
        return bt_search(n->left, v);
    return bt_search(n->right, v);
}
```



Complexity

Balanced: $O(\log n)$.

Sorted input (worst case): $O(n)$.



Tree Traversals: Pre-, In-, Post-order suffix

1st
Preorder

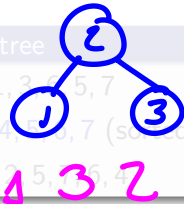
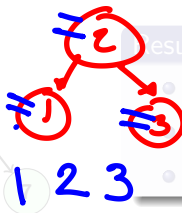
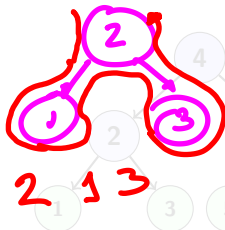
```
/*Root-Left-Right*/
void pre(BTreeNode *n)
{
    if (!n) return;
    visit(n); //print
    pre(n->left);
    pre(n->right);
}
```

2nd
Inorder

```
/*Left-Root-Right*/
void in(BTreeNode *n)
{
    if (!n) return;
    in(n->left);
    visit(n);
    in(n->right);
}
```

last
Postorder

```
/*Left-Right-Root*/
void post(BTreeNode *n)
{
    if (!n) return;
    post(n->left);
    post(n->right);
    visit(n);
}
```



Results on this tree

• Pre: 4, 2, 1, 3, 5, 7

• In: 1, 2, 3, 4, 5, 6, 7 (sorted!)

• Post: 1, 3, 2, 5, 4, 6, 2



Tree Traversals: Pre-, In-, Post-order

Preorder

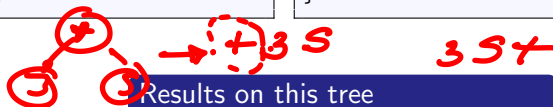
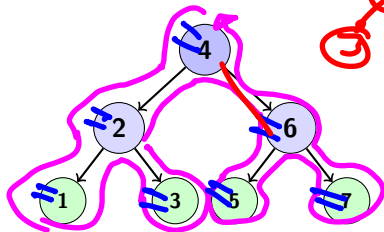
```
/*Root-Left-Right*/
void pre(BTNode *n)
{
    if (!n) return;
    visit(n);
    pre(n->left);
    pre(n->right);
}
```

Inorder

```
/*Left-Root-Right*/
void in(BTNode *n)
{
    if (!n) return;
    in(n->left);
    visit(n);
    in(n->right);
}
```

Postorder

```
/*Left-Right-Root*/
void post(BTNode *n)
{
    if (!n) return;
    post(n->left);
    post(n->right);
    visit(n);
}
```



Results on this tree

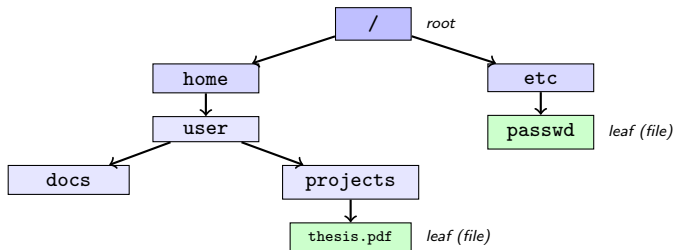
- **Pre:** 4, 2, 1, 3, 6, 5, 7
- **In:** 1, 2, 3, 4, 5, 6, 7 (sorted!)
- **Post:** 1, 3, 2, 5, 7, 6, 4



Study Case: File System as a Tree

Problem

Model a file system: directories contain files and sub-directories — a naturally **recursive** tree structure where path lookup is a root-to-node traversal.



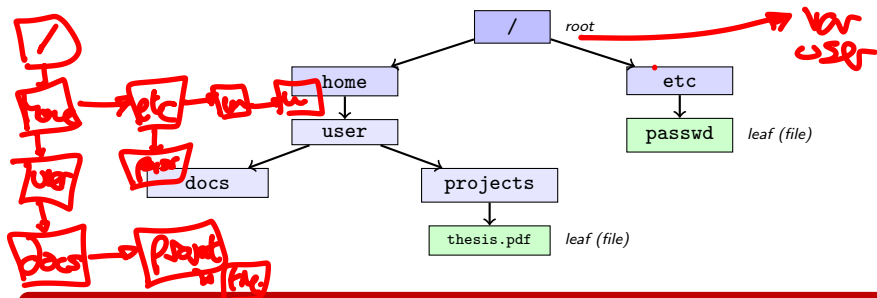
Why a Tree?

- Directories = internal nodes; files = leaves
- Postorder traversal computes total directory size

Study Case: File System as a Tree

Problem

Model a **file system**: directories contain files and sub-directories — a naturally **recursive** tree structure where path lookup is a root-to-node traversal.



Why a Tree?

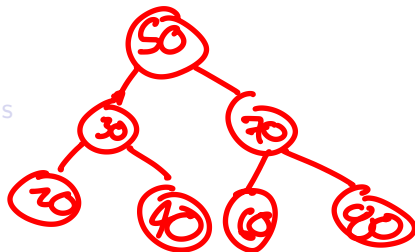
- Directories = internal nodes; files = leaves
- Postorder traversal computes total directory size

Outline

1 Tree Structures Fundamentals

2 **Balanced Trees**

3 Advanced Tree Structures



20 40 30 60 80 70 50



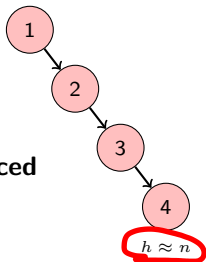
Why Balance Matters

1 2 3 4

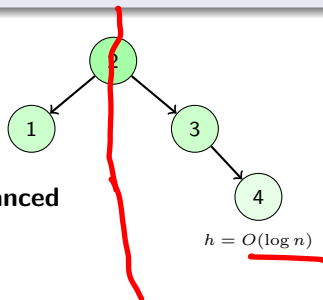
BST worst case

Inserting **sorted data** degrades the tree to a **linked list**: height $O(n)$ and all operations degrade from $O(\log n)$ to $O(n)$.

Unbalanced



Balanced



Unbalanced (sorted insert)

Height $\approx n \Rightarrow$ search $O(n)$

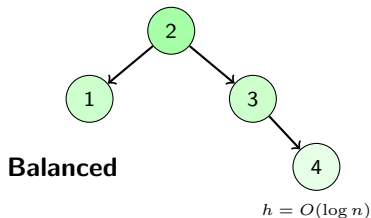
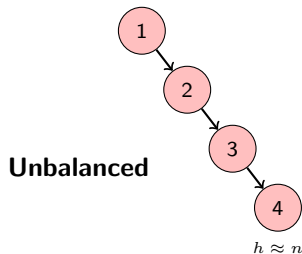
Balanced tree

Height $\approx \log_2 n \Rightarrow$ search $O(\log n)$

Why Balance Matters

BST worst case

Inserting **sorted data** degrades the tree to a linked list: height $O(n)$ and all operations degrade from $O(\log n)$ to $O(n)$.



Unbalanced (sorted insert)

Height $\approx n \Rightarrow$ search $O(n)$

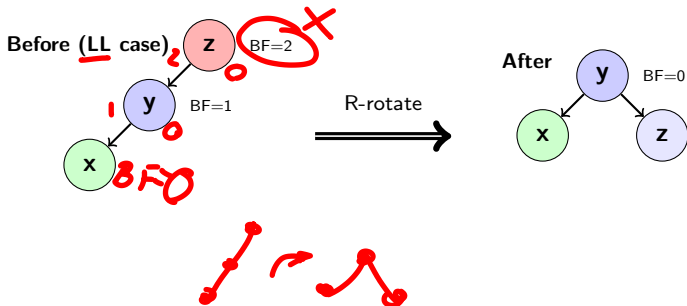
Balanced tree

Height $\approx \log_2 n \Rightarrow$ search $O(\log n)$

AVL Tree: Balance Factor & Rotations

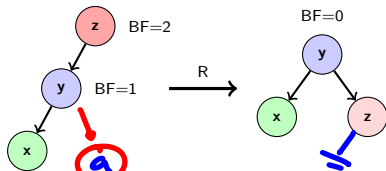
Definition

An AVL tree keeps $|h_{\text{left}} - h_{\text{right}}| \leq 1$ at every node. Violations are fixed immediately by rotations.

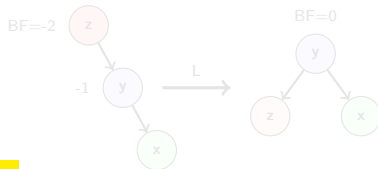


AVL: Four Rotation Cases Illustrated

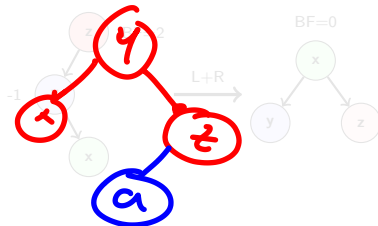
LL — single right rotation



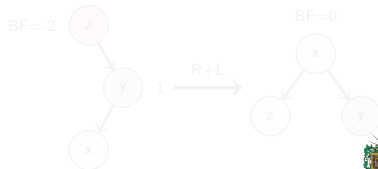
RR — single left rotation



LR — left on y , right on z

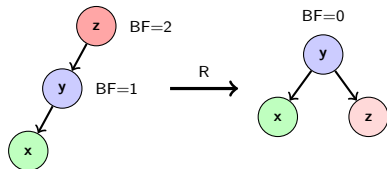


RL — right on y , left on z

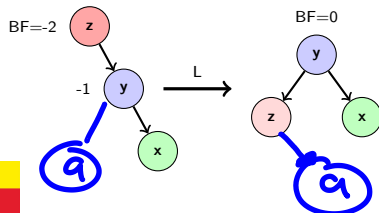


AVL: Four Rotation Cases Illustrated

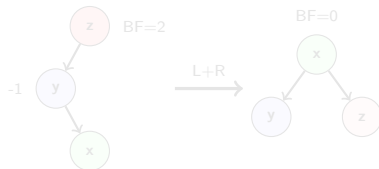
LL — single right rotation



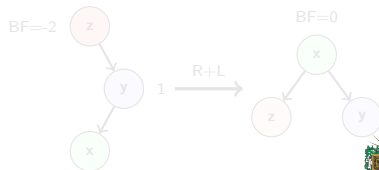
RR — single left rotation



LR — left on y , right on z

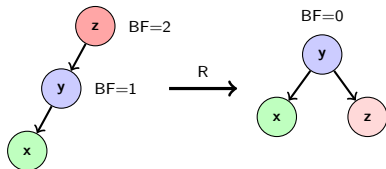


RL — right on y , left on z

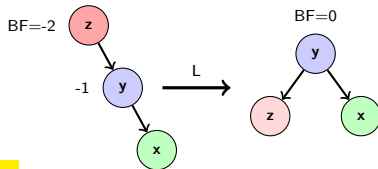


AVL: Four Rotation Cases Illustrated

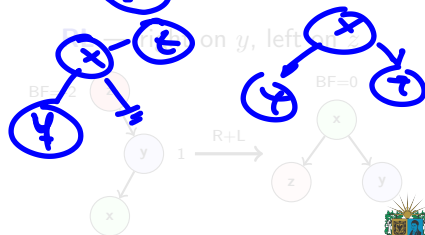
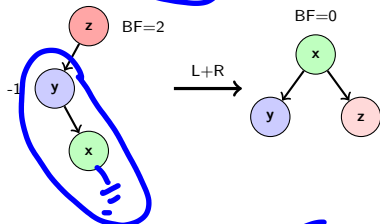
LL — single right rotation



RR — single left rotation

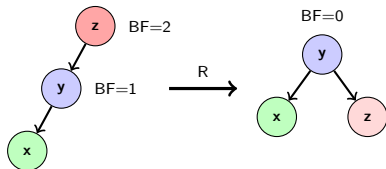


LR — left on y, right on z

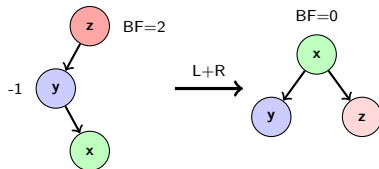


AVL: Four Rotation Cases Illustrated

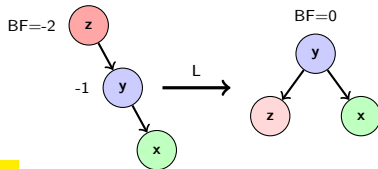
LL — single right rotation



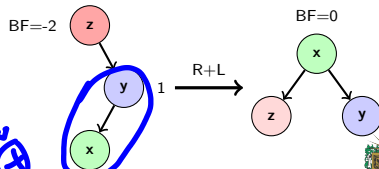
LR — left on y , right on z



RR — single left rotation



RL — right on y , left on z



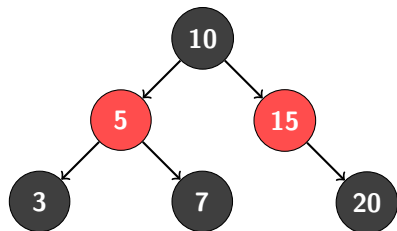
Red-Black Tree: Coloring Rules

Definition

A **Red-Black tree** is a BST where each node carries a **color** (red or black) satisfying four invariants that guarantee height $\leq 2 \log_2(n + 1)$.

Four invariants

- 1 Root is always **black**
- 2 All NIL sentinels are **black**
- 3 Red node $\Rightarrow$ both children are **black**
- 4 All paths root $\rightarrow$ leaf have the **same # of black nodes**



AVL vs. Red-Black: Comparison

Property	AVL	Red-Black
Max height	$1.44 \log n$	$2 \log n$
Search cost	Slightly faster	Good
Insert/Delete rotations	More frequent	Fewer
Memory overhead	Height field	1-bit color
Rebalance frequency	High	Lower

Choose AVL when

- Search-intensive workload
- More reads than writes
- Consistent $O(\log n)$ lookup needed

Choose Red-Black when

- Frequent inserts / deletes
- General-purpose map / set
- Used in C++ `std::map`



AVL vs. Red-Black: Comparison

Property	AVL	Red-Black
Max height	$1.44 \log n$	$2 \log n$
Search cost	Slightly faster	Good
Insert/Delete rotations	More frequent	Fewer
Memory overhead	Height field	1-bit color
Rebalance frequency	High	Lower

Choose AVL when

- Search-intensive workload
- More reads than writes
- Consistent $O(\log n)$ lookup needed

Choose Red-Black when

- Frequent inserts / deletes
- General-purpose map / set
- Used in C++ `std::map`



Outline

- 1 Tree Structures Fundamentals
- 2 Balanced Trees
- 3 Advanced Tree Structures



B-tree: Multi-key Nodes

Definition

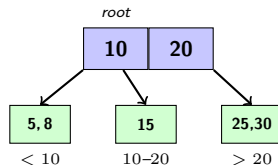
A **B-tree** of minimum degree t stores $t - 1$ to $2t - 1$ **keys per node**, keeping all leaves at the **same depth** — ideal for disk storage.

```
#define T 3    /* minimum degree */

typedef struct BNode {
    int      keys[2*T-1];
    struct BNode *child[2*T];
    int      n;      /* key count */
    int      leaf; /* boolean */
} BNode;

typedef struct { BNode *root; }
BTree;

BNode *btree_search(BNode*, int);
void    btree_insert(BTree*, int);
void    btree_delete(BTree*, int);
```



Why B-tree?

Wide branching $\Rightarrow$ few **disk reads**.

Used in NTFS, ext4, HFS+.

B-tree: Multi-key Nodes

Definition

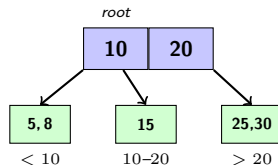
A **B-tree** of minimum degree t stores $t - 1$ to $2t - 1$ **keys per node**, keeping all leaves at the **same depth** — ideal for disk storage.

```
#define T 3    /* minimum degree */

typedef struct BNode {
    int      keys[2*T-1];
    struct BNode *child[2*T];
    int      n;      /* key count */
    int      leaf;   /* boolean */
} BNode;

typedef struct { BNode *root; }
BTree;

BNode *btree_search(BNode*, int);
void    btree_insert(BTree*, int);
void    btree_delete(BTree*, int);
```



Why B-tree?

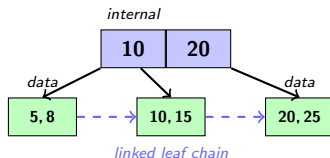
Wide branching $\Rightarrow$ few **disk reads**.

Used in NTFS, ext4, HFS+.

B+ Tree: Leaf Linking

Definition

A **B+ tree** stores **all data at leaves**; internal nodes hold only **routing keys**. Leaves are **linked** in a sorted chain for efficient range scans.



B+ advantages over B-tree

- Range queries: follow leaf links
- Full scans: traverse leaf level only
- All search paths same length

Databases

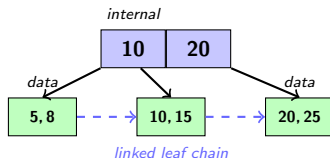
MySQL InnoDB, PostgreSQL, Oracle — all use B+ trees for primary indexes.



B+ Tree: Leaf Linking

Definition

A **B+ tree** stores **all data at leaves**; internal nodes hold only **routing keys**. Leaves are **linked** in a sorted chain for efficient range scans.



B+ advantages over B-tree

- Range queries: follow **leaf links**
- Full scans: traverse **leaf level only**
- All search paths **same length**

Databases

MySQL InnoDB, PostgreSQL, Oracle — all use B+ trees for primary indexes.



Outline

- 1 Tree Structures Fundamentals
- 2 Balanced Trees
- 3 Advanced Tree Structures



Complexity Summary

Structure	Search	Ins./Del.	Best for
BST (avg)	$O(\log n)$	$O(\log n)$	General purpose
BST (worst)	$O(n)$	$O(n)$	Avoid sorted input
AVL	$O(\log n)$	$O(\log n)$	Read-heavy apps
Red-Black	$O(\log n)$	$O(\log n)$	Mixed read/write
B-tree	$O(\log_t n)$	$O(\log_t n)$	File systems
B+ tree	$O(\log_t n)$	$O(\log_t n)$	DB indexes + scans

Space complexity

All structures: $O(n)$ for n keys.

Node overhead: **2 pointers** (binary trees) vs. **$2t$ pointers** (B-tree of degree t).



Outline

- 1 Tree Structures Fundamentals
- 2 Balanced Trees
- 3 Advanced Tree Structures



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-i>

